

Model Prediction

Model Prediction

- 1、Optimal Performance Mode
 - 1.1、Enable MAX Mode
 - 1.2、Enable Jetson Clocks
- 2、Model Prediction
 - 2.1、CLI Usage
 - 2.2、Python Usage
 - Effect Preview

1、Optimal Performance Mode

1.1、Enable MAX Mode

Enabling MAX Power Mode on Jetson will ensure all CPU and GPU cores are enabled:

```
sudo nvpmode1 -m 0
```

1.2、Enable Jetson Clocks

Enabling Jetson Clocks will ensure all CPU and GPU cores run at maximum frequency:

```
sudo jetson_clocks
```

2、Model Prediction

Note: This example requires first running the "Model Training_Conversion" course to generate model files before proceeding.

2.1、CLI Usage

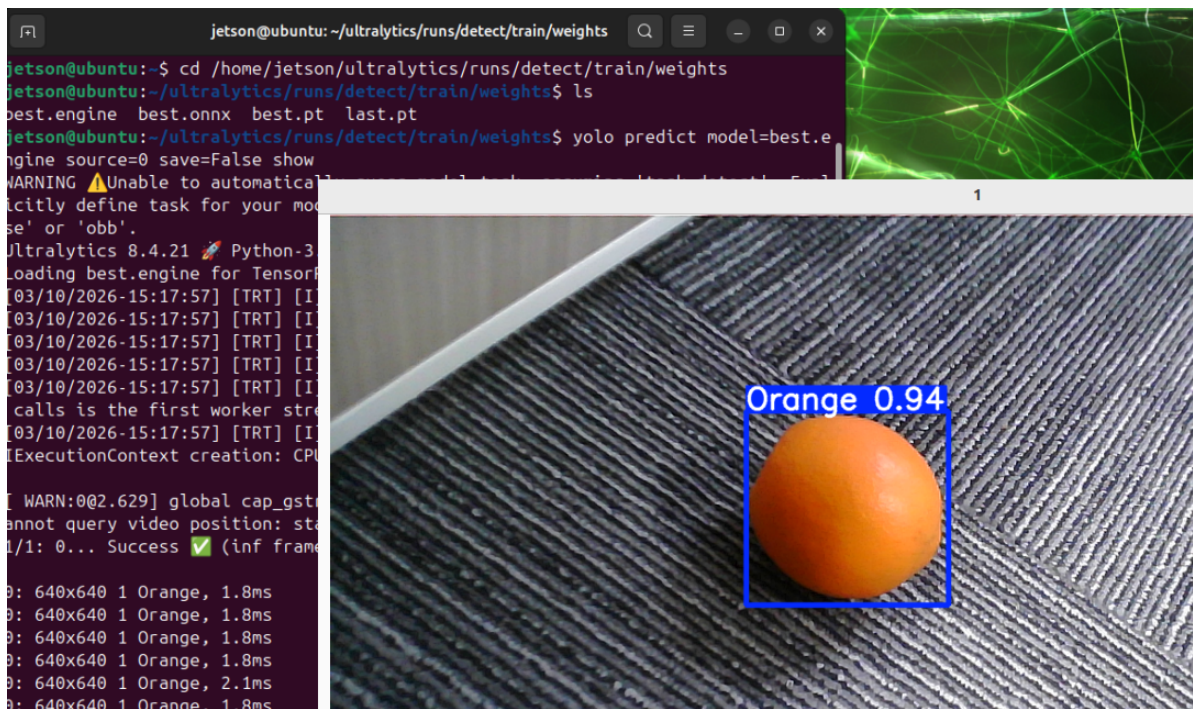
CLI currently only supports calling USB cameras.

Open terminal, enter the path generated by the previous course model

```
cd  
/home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/runs/detect/train/weights
```

Start the program

```
yolo predict model=best.engine source=0 save=False show
```



2.2、Python Usage

Please first insert the USB camera into the USB port of the board, then use the Python program to call the USB camera to recognize oranges.

Use best.engine to predict camera footage:

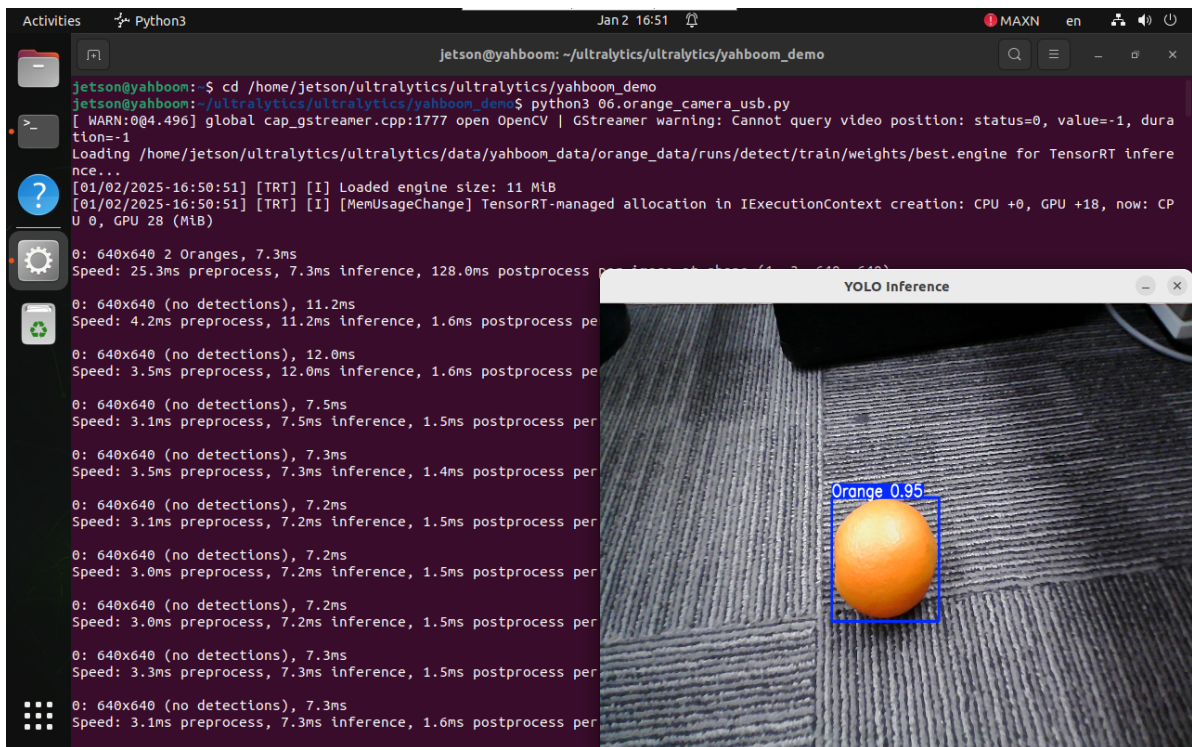
```
cd /home/jetson/ultralytics/ultralytics/yahboom_yolo26
```

Run code: Click on the preview window and press 'q' key to terminate the program!

```
python3 06.orange_camera_usb.py
```

Effect Preview

Point the camera at the orange, the program will automatically frame the orange and display the score.



Sample code:

```

import cv2
from ultralytics import YOLO

# Load the YOLO model
# model =
YOLO("/home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/runs/detect/train/weights/best.pt")
# model =
YOLO("/home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/runs/detect/train/weights/best.onnx")
model =
YOLO("/home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/runs/detect/train/weights/best.engine")

# Open the camera
cap = cv2.VideoCapture(0)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path =
"/home/jetson/ultralytics/ultralytics/output/06.orange_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

```

```

if success:
    # Run YOLO inference on the frame
    results = model(frame)

    # Visualize the results on the frame
    annotated_frame = results[0].plot()

    # Write the annotated frame to the output video file
    out.write(annotated_frame)

    # Display the annotated frame
    cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

    # Break the loop if 'q' is pressed
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
else:
    # Break the loop if the end of the video is reached
    break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

Note: In the code, the path pointing to the model `model = YOLO(path)`, if the model data is retrained and updated, the path needs to be replaced with your latest path.